

立项报告

第三十八组

March 2026

1 可行性分析报告

1.1 引言

1. 编写目的

本报告旨在对”基于 Java 的 AI 辅助外卖平台”项目进行系统性可行性研究，从技术、经济、社会三个维度评估项目的实施价值与落地条件，为项目的正式立项决策提供科学依据。

本报告的预期读者为课程指导教师及项目评审人员。

2. 项目背景

随着移动互联网的普及，在线外卖已成为城市居民日常餐饮消费的重要方式。然而，现有外卖平台普遍存在信息过载的问题——用户面对海量商家与菜品时，往往需要耗费大量时间进行筛选比较，决策效率低下。

与此同时，以大语言模型 (LLM) 为核心的 AI Agent 技术近年来快速成熟，具备理解自然语言、调用外部工具、完成多步骤推理的能力，为解决上述问题提供了新的技术路径。

基于此背景，本项目拟在构建完整外卖平台基础功能的同时，引入 AI 点餐助手模块。用户可以通过自然语言描述口味偏好与预算需求，由 Agent 自动完成商家检索、菜品筛选与多维度比较，最终给出个性化推荐方案，实现”货比三家”的智能辅助点餐体验。

本项目作为课程实践作业，旨在综合运用 Java 后端开发、微服务架构与 AI 工程化集成等技术，完成一个具备实际应用价值的完整软件系统。

3. 定义与术语

术语 / 缩略语	说明
LLM	Large Language Model, 大语言模型，指具备自然语言理解与生成能力的 AI 模型，如 DeepSeek、通义千问等
Agent	基于 LLM 的智能体，能够理解用户意图、自主规划并调用外部工具完成复杂任务

接下页...

术语 / 缩略语	说明 (续)
Tool Calling	工具调用机制, LLM 在推理过程中识别需要调用的函数并传入参数, 由程序执行后将结果返回给模型
LangChain4j	Java 生态下的 LLM 应用开发框架, 提供 Agent、Tool、Memory 等高级抽象
JWT	JSON Web Token, 一种无状态的身份认证令牌标准
MVP	Minimum Viable Product, 最小可行产品, 指以最少功能满足核心需求的版本
API	Application Programming Interface, 应用程序编程接口

4. 参考资料

- (a) 课程讲义:《立项报告撰写说明》;
- (b) LangChain4j 官方文档. <https://docs.langchain4j.dev>
- (c) Spring Boot 官方文档. <https://spring.io/projects/spring-boot>
- (d) DeepSeek 开放平台 API 文档. <https://platform.deepseek.com/docs>

1.2 项目概述

1. 项目目标

本项目旨在设计并实现一个基于 Java 的 AI 辅助外卖平台, 核心目标如下:

基础目标: 构建一个功能完整的外卖平台, 支持商家端的店铺与菜品管理, 以及用户端的浏览、下单全流程, 满足课程 Java 项目的基本要求。

创新目标: 在基础平台之上集成 AI 点餐助手模块, 用户可通过自然语言描述口味偏好、预算范围等需求, 由 Agent 自动完成多商家检索、菜品筛选与综合比较, 输出个性化推荐方案, 显著降低用户的点餐决策成本。

2. 项目范围

系统包含以下功能:

商家端 (Web 网页管理后台): 支持商家注册、店铺基本信息管理 (营业时间、配送范围等)、菜品的上架与下架、价格修改及库存管理。

用户端 (移动端小程序): 支持用户注册与登录、基于地理位置的附近商家浏览与关键词搜索、购物车与订单提交、模拟支付与订单状态查询, 以及核心的 AI 点餐助手功能——用户输入需求, 系统返回商家推荐结果。

系统不包含以下内容:

真实第三方支付功能、骑手配送调度系统、商家营业数据统计、用户评分体系。上述功能超出课程项目范围, 可作为软件更新升级方向。

3. 用户群体

本系统面向两类用户群体：

商家用户：有进行外卖业务需求的餐饮商家，核心诉求是便捷地管理店铺信息与菜品数据，确保线上展示内容准确及时。

消费者用户：具备点外卖习惯的普通用户，核心诉求是快速找到符合自身口味与预算的餐食选项。其中部分用户面对海量商家时存在选择困难，是 AI 点餐助手功能的主要目标受众。

4. 业务价值

本项目的业务价值体现在两个层面：

平台价值：通过数字化手段连接商家与消费者，商家得以低成本地开展外卖业务，消费者得以便捷浏览附近餐食并完成下单，实现供需双方的高效匹配。

AI 业务价值：传统外卖平台的推荐机制以人工排序和广告投放为主，难以真正理解用户的个性化需求。本项目引入的 AI 点餐助手基于用户自然语言输入，结合历史偏好与实时菜品，完成跨商家的客观比较，为用户提供更具参考价值的决策辅助，体现了 AI 技术在现实生活场景落地的应用价值。

1.3 候选技术方案

1.3.1 方案一：单体架构

1. 方案概述

方案一采用经典单体架构，将用户管理、商家管理、菜品管理、订单处理、AI 点餐助手等所有功能模块统一集成在一个 Spring Boot 应用中，以单一进程对外提供服务。该方案以工程简洁性为优先原则，适合团队规模小、开发周期短的课程项目场景。

2. 技术架构

系统整体采用前后端分离架构，前端根据业务场景分为用户端（小程序）与商家端（Web 管理系统）。两端均通过 RESTful API 及 HTTP 请求访问后端统一入口。后端按 Controller → Service → Mapper 分三层来处理业务，其中热点数据缓存至 Redis 中。AI 点餐助手作为独立的 Service 模块内嵌于同一应用中，通过 LangChain4j 调用外部 LLM API 完成推理。

3. 开发工具与平台

层次	技术选型
后端框架	Spring Boot 3、MyBatis-Plus
数据库	MySQL 8.0
缓存	Redis 7
AI 集成	LangChain4j
身份认证	JWT + Spring Security

接下页...

层次	技术选型 (续)
前端框架 (商家端)	Vue 3 + React
前端框架 (用户端)	Uni-app
构建工具	Maven
开发环境	IntelliJ IDEA、JDK 17

4. 第三方依赖

依赖组件	说明及许可协议
DeepSeek API	提供核心的自然语言处理与大语言模型推理能力。
LangChain4j	用于构建 AI Agent 及整合大模型 API，采用 Apache 2.0 开源许可协议。
MyBatis-Plus	用于简化数据库持久层操作，采用 Apache 2.0 开源许可协议。

5. 人员配置要求

本项目作为课程实践，且采用单体架构，工程复杂度适中。主要需要 1-2 名具备扎实 Java 后端基础的人员。开发人员需熟悉 Spring Boot 生态，掌握 MySQL 与 Redis 等中间件的日常开发，并能完成 API 接口与 AI 模块的联动。另需要一名有前端开发经验的人员，能实现前后端的联动、调用。

6. 实施周期预估

阶段	内容	预估时长
需求与设计	数据库建表、接口设计	1 周
基础平台开发	商家端 + 用户端主链路	2 周
Agent 模块集成	工具函数开发、LLM 接入	1 周
测试与文档	进行软件测试、文档整理	1 周
合计		5 周

1.3.2 方案二：微服务架构

1. 方案概述

方案二在方案一的功能基础上，采用微服务架构对系统进行拆分，将用户服务、商家服务、订单服务、AI Agent 服务拆分为独立部署的进程，通过 Spring Cloud 套件实现服务治理。该方案工程复杂度更高，但具备更好的模块独立性与扩展能力。

2. 技术架构

前端请求统一经由 Spring Cloud Gateway 网关进入，网关负责路由转发。各微服务注册至 Nacos，服务间调用通过 OpenFeign 完成。每个服务独立连接数据库，AI Agent 服务作为独立进程，专门处理自然语言点餐请求并调用其他服务的查询接口获取数据。

3. 开发工具与平台在方案一开发工具的基础上，额外引入：

层次	技术选型
后端框架	Spring Cloud 2023
服务间调用	OpenFeign

4. 第三方依赖

在方案一依赖的基础上，额外引入：

依赖组件	说明及许可协议
Spring Cloud Gateway	提供统一 API 网关，采用 Apache 2.0 开源许可协议。。
Nacos	作为服务注册与配置中心，采用 Apache 2.0 开源许可协议。。
RabbitMQ	用于订单异步消息处理，采用 Apache 2.0 开源许可协议。。

5. 人员配置要求

除方案一所需技能外，还需额外掌握 Spring Cloud 组件配置、多服务本地联调方法及分布式环境下的问题排查能力，技术门槛显著高于方案一，预计需要 4-6 人的研发团队。

6. 实施周期预估

阶段	内容	预估时长
需求与设计	服务拆分设计、数据库建表	1 周
基础设施搭建	Gateway、Nacos、RabbitMQ 配置	2 周
各服务开发	用户/商家/订单服务主链路	3 周
Agent 服务集成	工具函数开发、LLM 接入	1 周
测试与文档	多服务联调测试、文档整理	1 周
合计		8 周

1.4 可行性分析

本节将从技术、经济和社会三个维度，对本项目的两个候选技术方案进行系统性评估，以验证项目正式立项的可行性。

1.4.1 技术可行性

1. 技术成熟度

两个候选方案所涉及的技术栈均具备高度的工业级成熟度。在前端表现形态上，用户端小程序采用的 Uni-app（基于 Vue.js）与商家端采用的 Vue 网页管理后台，均拥有极其成熟的组件库与跨端编译能力。后端方面，方案一依托的 Spring Boot 与 MyBatis-Plus，以及方案二依托的 Spring Cloud 分布式组件均已得到广泛验证。AI 模块依赖的 LangChain4j 框架及 DeepSeek API 也具备商用级的稳定性。

2. 资源可用性

软件、硬件与数据资源对两方案均可用，但在人力资源约束上差异显著。方案一工程复杂度适中，1-2 名开发人员即可胜任全栈开发；方案二实施门槛较高，预计需要 4-6 人的团队支撑，与本团队成员数（两人）不符。

3. 系统性能与约束

方案一能够完全满足课程级别的并发访问与响应时间约束，系统开销小；方案二具备更高的扩展性，但跨服务 RPC 调用会增加网络延迟。两个方案共同面临的性能约束在于调用大语言模型（LLM）API 时的推理耗时，此问题可通过前端加载动画与异步状态轮询等工程手段加以缓解。

4. 技术风险

方案一的技术风险主要集中在 AI Agent 多步推理过程中的幻觉现象及 API 调用的偶发不稳定，可通过设计严谨的 Prompt 模板与兜底机制规避。方案二除包含上述 AI 风险外，还面临较高的架构级风险，如分布式事务数据不一致、多服务本地联调困难等，容易导致项目开发严重延期。

1.4.2 经济可行性

1. 成本估算

本项目的核心成本为研发团队的时间投入，直接资金成本极低。方案一仅需租赁单台轻量级云服务器与少量大模型 API Token 费用，整体资金成本可控制在 100 元以内。方案二因引入了网关、注册中心、消息队列等诸多基础组件，需要更高配置的服务器集群作为底层支撑，整体资金成本会略高于方案一，但总体资金成本在 500 元范围内。

2. 效益预测

本项目的商业变现途径主要为商家端抽成佣金、以及平台广告展示费。本项目引入的 AI 点餐助手作为核心差异化卖点，能够有效解决用户“选择困难”的痛点，预期将显著提升平台的订单转化率、客单价与用户黏性，从而为平台创造出高于传统外卖系统的市场经济效益。

3. 投资回报分析

方案一前期资金投入少，能够以最快速度推向市场进行商业模式验证，预计投资回报期较短，资金周转效率高。方案二系统架构的理论承载上限很高，但在平台业务起步、用户访问量尚未爆发的早期阶段，过度庞大的架构会造成严重的服务器资源闲置；前期投入的沉没成本较高，导致投资回报期被拉长，整体收益率表现一般。

4. 敏感性分析

方案一对初期市场收益和用户增长率波动的敏感度较低，即便前期市场拓展速度未达预期，其低廉的日常运维支出也允许企业拥有充足的试错时间和资金缓冲期。方案二则对开发周期与市场收益较为敏感，若遭遇微服务联调导致的开发延期，较高的固定硬件开支与团队人力消耗可能带来资金的亏损。

1.4.3 社会可行性

1. 法律合规性

项目中使用的软件开发工具、框架均遵循 Apache 2.0 等开源许可协议，不存在知识产权和版权纠纷风险。系统开发与测试期间均使用自主构造的模拟商家和模拟菜品数据，不涉及爬取真实商家的商业数据，且不接入真实的第三方支付系统。

2. 社会伦理影响

本项目引入的 AI 点餐助手旨在根据用户输入的口味与预算提供客观的菜品筛选与比较，不涉及任何基于用户身份、地域的算法偏见或歧视。同时，精准匹配用户胃口与预算的智能推荐，能够在一定程度上减少盲目点餐造成的食物浪费，产生积极的社会引导意义。

3. 组织适应性

本项目的组织形式为敏捷型的小规模学生开发团队。方案一秉持单体架构的极简与高效，与这种短周期的组织高度契合；而方案二工程体量较为庞大，略微超出了当前学生团队的组织适应性范围。

1.5 方案比较与推荐

在完成上述各项可行性评估的基础上，本节将对方案一（单体架构）与方案二（微服务架构）进行综合比较，并明确提出本项目的最终技术选型。

1.5.1 比较维度与综合评估

下面报告将从技术风险、经济收益、实施周期、社会合规性等四个核心维度，对两个候选方案进行横向对比。

比较维度	方案一：单体架构（前后端分离）	方案二：微服务架构
技术风险	低。技术栈成熟，单进程内排错直观，AI 接口联调难度可控。	高。涉及复杂的分布式事务一致性、网络链路追踪、服务雪崩风险。
实施周期	约 5 周。开发周期短，适合敏捷迭代。	约 8 周。前期基建耗时长，微服务联调易导致延期。
经济成本	低。单机云服务器即可运行。	中。需高可用集群支撑。
系统扩展性	中。受限于单节点性能上限，只能进行整体纵向扩容。	高。支持按业务领域（如订单、用户）进行细粒度的横向扩容。

接下页...

比较维度	方案一：单体架构（前后端分离）（续）	方案二：微服务架构（续）
项目契合度	高。能够将核心精力集中于 AI Agent 业务逻辑的落地验证。	低。研发精力易被庞大的微服务基础设施搭建所严重分散。

1.5.2 推荐方案

基于上述综合评估结果，本项目明确推荐采用方案一：单体架构。

1.5.3 推荐理由

推荐该方案的理由如下：

1. 该方案契合快速商业验证的逻辑，能够以最快的速度交付 MVP。规避了微服务在业务早期的过度设计。
2. 更低的资金投入，使得团队在项目早期的开发中不会被庞杂的底层基建所分散，而是专注于核心业务。
3. 人力资源消耗与团队现状匹配，最大程度上避免了因人手不足导致的开发延期。

1.6 风险分析与应对建议

本部分对项目可能面临的各类风险进行系统识别，并提出应对建议。

1.6.1 风险识别

结合本项目的现状，我们从技术、管理、需求、人员、外部环境等五个维度识别出以下潜在风险：

1. 技术风险（T1）：LLM “幻觉” 风险。AI Agent 在进行 Tool Calling 或多步推理时，可能生成不存在的菜品或传递错误的参数。
2. 技术风险（T2）：跨平台前端兼容性风险。使用 Uni-app 编译小程序时，部分原生组件或 CSS 样式在不同端可能出现渲染不一致的问题。
3. 管理风险（M1）：项目进度延期风险。由于核心前后端联调及 LangChain4j 的 AI 集成环节存在不确定性，可能导致关键里程碑逾期。
4. 需求风险（R1）：AI 交互边界模糊导致的需求变更风险。项目初期对“AI 辅助点餐”的智能化程度定义可能不清晰（例如：是单轮问答还是多轮复杂对话？是否包含深度的个性化推荐？）。若在开发中途预期拔高，将导致底层数据结构和 LangChain4j 模块的重构与返工。
5. 人员风险（P1）：前端技术栈生疏风险。团队成员主要精力集中在 Java 后端，对 Vue 及 Uni-app 的实操经验相对欠缺，可能在页面开发上耗费过多时间。
6. 外部环境风险（E1）：第三方 API 服务不稳定风险。DeepSeek API 可能因网络波动、平台政策调整或账户限流导致接口调用超时或不可用。

1.6.2 风险评估

对上述识别出的风险，我们进行了发生概率和影响程度的评估，具体如下表所示：

风险编号	风险描述	发生概率	影响程度
T1	大模型“幻觉”与参数传递错误	高	高
R1	AI 交互边界模糊导致的需求变更	中	高
M1	接口联调与 AI 集成导致进度延期	中	高
E1	第三方 API 服务不稳定或限流	低	高
P1	前端技术栈生疏导致开发效率低	中	中
T2	Uni-app 跨端样式渲染不兼容	低	低

1.6.3 应对策略

针对上述评估出的高、中优先级风险，我们提出以下具体的应对策略：

1. 针对大模型“幻觉”风险（T1）——风险缓解：

在 LangChain4j 的开发中，采用严谨的提示词工程，严格限制 System Message。要求 Agent 的输出必须遵循预定义的 JSON Schema。同时在后端业务层增加参数合法性校验拦截器，若识别到非法参数，则启动降级策略（如返回默认的热销菜品列表）。

2. 针对 AI 交互边界模糊风险（R1）——风险规避：

在需求规格说明阶段，严格界定 MVP 中 AI Agent 的能力边界。明确规定本期核心仅实现“基于自然语言意图识别的条件检索”，暂不引入多轮对话记忆与复杂的用户画像分析。通过与评审方提前对齐验收标准，切断后期需求变更的可能性。

3. 针对项目进度延期与前端经验欠缺风险（M1, P1）——风险缓解与接受：

接受团队在前端定制化设计上的短板，直接引入成熟的开源 UI 组件库，避免从零手写样式。优先跑通“前后端基础交互”与“AI 接口调用”两条主干链路，确保核心点餐流程按期交付。

1.7 结论与建议

1.7.1 可行性结论

基于上文的分析，对“基于 Java 的 AI 辅助外卖平台”项目的可行性结论为：**可行**。

本项目在技术、经济、社会三方面均具备充分的可行性，建议正式立项。该方案能够在软硬件资源受限、研发团队规模较小的客观条件下，以最低的试错成本完成 MVP 的交付。它能够切实验证“大语言模型智能辅助点餐”的创新业务价值，同时为团队带来扎实的高并发场景工程经验。

1.7.2 后续工作建议

若项目正式启动，针对后续阶段提出以下重点工作建议：

1. 开展产品原型设计与验证：建议立项后一周内，优先着手绘制用户端小程序与商家 Web 管理后台的 UI 原型，明确页面流转逻辑，特别需要确定 AI 点餐助手的交互边界。
2. 基础架构搭建：根据前期的资源预估，尽快完成云端轻量应用服务器的租赁、MySQL 与 Redis 环境的部署，以及 Spring Boot 后端工程的骨架初始化。
3. 启动需求分析：在立项后一周内，基于已经组建完成项目小组的现状，完成需求分析。

2 项目立项报告

2.1 项目范围与可交付成果

2.1.1 项目目标

本项目拟开发一个基于 Java 的 AI 辅助外卖平台，面向普通消费者与餐饮商家提供基础外卖业务支持，并在传统外卖平台功能基础上引入 AI 点餐助手，以提升用户点餐决策效率。项目目标分为基础目标与创新目标两个层面。

基础目标是构建一个功能完整的外卖平台原型系统，实现商家端店铺与菜品管理、用户端浏览与下单等核心业务流程，满足课程项目对 Java 软件系统完整性的要求。创新目标是在基础平台之上集成 AI 点餐助手模块，使用户能够通过自然语言输入口味偏好、预算范围等需求，由系统自动完成商家检索、菜品筛选与综合比较，并返回个性化推荐结果，从而降低用户在面对大量商家与菜品时的选择成本。

2.1.2 项目建设范围

本项目的建设范围主要包括商家端、用户端、后端业务服务以及 AI 点餐助手四个部分。

商家端主要面向入驻平台的餐饮商家，提供店铺信息维护与菜品管理功能，具体包括商家注册、店铺基本信息管理、营业时间设置、配送范围设置、菜品上架与下架、价格修改以及库存管理等功能。用户端主要面向普通消费者，提供注册登录、附近商家浏览、关键词检索、购物车管理、订单提交、模拟支付与订单状态查询等核心功能，并支持用户通过自然语言方式调用 AI 点餐助手获取推荐结果。后端业务服务作为系统核心，负责用户管理、商家管理、菜品管理、订单处理、权限认证及数据存储等业务逻辑实现。AI 点餐助手模块负责接收用户的自然语言输入，结合商家与菜品数据完成条件筛选和推荐生成，为用户提供辅助点餐服务。

2.1.3 主要可交付成果

本项目在实施完成后应形成以下主要可交付成果：

1. 后端程序系统，包括用户管理、商家管理、菜品管理、订单处理、身份认证等核心服务模块；
2. 商家端前端系统，用于商家完成店铺与菜品管理；
3. 用户端前端系统，用于用户完成浏览、下单及 AI 辅助点餐交互；
4. AI 点餐助手模块，实现自然语言输入、条件解析、商家与菜品筛选及推荐结果返回；

5. 测试与部署相关材料，包括系统测试记录、运行说明和项目总结文档。

2.2 技术路线与实施方案

2.2.1 技术路线

本项目采用前后端分离的单体架构。系统前端分为商家端 Web 管理系统和用户端移动端小程序，两端通过 RESTful API 与后端统一服务进行交互。后端以 Java 为核心开发语言，基于 Spring Boot 构建统一业务应用。数据存储层采用 MySQL，热点数据缓存采用 Redis，AI 能力通过 LangChain4j 对接外部大语言模型接口完成。该技术路线能够在保证系统完整性的同时，兼顾开发效率与实现难度。

2.2.2 实施方案

在具体实施上，本项目首先完成需求分析、数据库设计及后端基础工程搭建，建立系统运行骨架；其次实现商家端与用户端的核心业务主链路，确保平台具备基本的浏览、下单和管理能力；随后接入 AI 点餐助手模块，完成自然语言输入解析、商家与菜品检索及推荐结果返回；最后进行测试与文档整理。

2.3 项目资源计划

2.3.1 人员资源

项目实施所需资源以人员资源为主，团队成员共两人。本项目采用单体架构，整体工程复杂度适中，人员配置以小规模协作开发为主。团队成员需具备 Java 后端开发基础，能够完成 Spring Boot 项目搭建、数据库设计、接口开发及业务逻辑实现。同时，还需具备一定的前端开发能力。在 AI 模块方面，开发人员还需具备基本的大模型接口调用与提示词设计能力。

2.3.2 硬件资源

本项目对硬件资源的要求较低。开发阶段主要依赖团队成员现有个人计算机完成代码编写、调试与测试工作，不需要额外采购专用硬件设备。系统部署阶段可采用单台轻量级云服务器，用于运行后端服务、数据库及缓存组件。

2.3.3 软件与工具资源

本项目后端开发采用 Spring Boot、MyBatis-Plus、MySQL 和 Redis 等成熟技术组件，前端开发使用 Web 前端框架与 Uni-app，AI 模块集成采用 LangChain4j，并通过外部大语言模型接口提供自然语言处理能力。项目构建采用 Maven，开发环境采用 IntelliJ IDEA 与 JDK 17。此外，项目开发过程中还需使用 Git 等版本管理工具。

2.3.4 外部服务资源

本项目在实施过程中还依赖一定的外部服务资源，主要包括大语言模型 API 服务以及相关开源组件支持。

2.4 工作量估算

2.4.1 软件规模估算

结合本项目的功能划分，可将系统拆分为用户管理、商家管理、菜品管理、订单处理、AI 点餐助手、身份认证与权限控制、前端页面与接口联调等几个主要模块。根据各模块的实现内容，对核心代码规模估算如下：

功能模块	预计代码规模 (LOC)
用户管理模块	130
商家管理模块	200
菜品管理模块	200
订单处理模块	240
AI 点餐助手模块	160
身份认证与权限控制模块	100
前端页面与接口联调	220
测试与辅助代码	100
合计	1350

因此，本项目预计核心开发代码规模约为 1350 LOC。从规模上看，该项目低于课程说明中组织型项目的上限要求，且整体功能边界较为清晰，适合采用组织型 COCOMO 参数进行后续估算。

2.4.2 工作量估算

根据基本 COCOMO 模型，组织型项目的工作量估算公式为：

$$E = a \times (KLOC)^b$$

其中，组织型项目参数取 $a = 2.04, b = 1.05$ ，将本项目规模 $KLOC = 1.5$ 带入，有：

$$E = 2.4 \times (1.35)^{1.05} \approx 3.29$$

因此，本项目的估算工作量约为 3.29 人月，该结果表明，在严格控制项目范围、优先完成核心功能的前提下，本项目的开发任务量处于可接受范围内。

2.4.3 结果分析

从估算结果来看，本项目整体规模较小，符合课程项目以最小可行产品为导向的实施特征。综合来看，本项目的软件规模与工作量均与前文确定的单体架构方案、人员配置方式和实施周期保持一致，具备较好的可执行性。

2.5 成本估算

2.5.1 成本构成

本项目的成本主要由人员投入成本、软硬件资源成本和外部服务成本三部分构成。其中，人员投入成本是项目实施过程中的主要成本来源。软硬件资源成本主要包括项目运行所需的轻量级云服务器及基本开发环境。外部服务成本主要体现在大语言模型 API 的调用费用。由于本项目为课程实践项目，开发设备以团队现有个人计算机为主，开发工具大多采用开源或免费软件，因此整体资金支出较低。

2.5.2 直接成本估算

结合项目的实际需求，本项目的直接资金成本估算如下：

成本项目	估算金额（元）	说明
云服务器租用	50	用于部署后端服务及运行演示
大语言模型 API 调用	30	用于 AI 点餐助手功能测试与展示
其他辅助支出	20	包括网络、调试及少量临时支出
合计	100	

2.5.3 间接成本估算

除直接资金支出外，本项目还存在一定的间接成本。由于项目涉及前后端协同开发以及 AI 模块集成，团队成员需要投入一定时间熟悉相关技术栈。此外，项目开发过程中还需要进行需求讨论、任务分配和文档整理，这些内容虽然不直接产生资金支出，但会占用一定的开发时间。总体来看，项目的间接成本主要体现在人力与时间投入层面，但在两人协作开发的条件下仍处于可接受范围内。

2.5.4 成本分析

综合分析可知，本项目的成本结构较为简单，且整体支出水平较低。由于系统采用单体架构，避免了微服务带来的额外运维开销；同时使用成熟开源框架和模拟数据进行开发，也进一步降低了实施成本。

2.6 项目进度安排

2.6.1 项目进度阶段划分

结合项目实施内容与工作量估算结果，本项目拟划分为以下五个阶段：

1. 需求分析与系统设计阶段：完成需求细化、功能模块划分、数据库初步设计和系统接口设计；
2. 基础平台开发阶段：完成商家端与用户端核心业务流程的后端实现，以及前端基础页面搭建；
3. 前后端联调阶段：完成用户注册登录、商家管理、菜品管理、订单处理等主链路联调；
4. AI 模块集成阶段：完成 AI 点餐助手的接口接入、提示词设计、条件检索与推荐结果返回；

5. 测试与文档整理阶段：完成系统测试、问题修复、文档整理及答辩材料准备。

可以发现，上述阶段之间具有较强的先后依赖关系。

2.6.2 时间安排

根据项目实际情况，结合两人协作开发模式，本项目计划总开发周期为 6 周。具体安排如下表所示：

阶段	主要内容	时间安排
需求分析与系统设计	需求细化、数据库设计、接口设计	第 1 周
基础平台开发	后端基础模块开发、前端基础页面搭建	第 2-3 周
前后端联调	主链路联调与功能完善	第 4 周
AI 模块集成	LLM 接入、提示词设计、推荐功能实现	第 5 周
测试与文档整理	系统测试、问题修复、文档与答辩材料整理	第 6 周

该时间安排在总体上与项目规模、人员配置和实施目标相匹配，能够较好地覆盖系统开发的主要过程。

2.6.3 甘特图

该项目所对应的甘特图如下：

表 13: 图 2-1 项目进度甘特图

任务阶段 \ 时间 (周 a'a)						
	第 3 周	第 4 周	第 5 周	第 6 周	第 7 周	第 8 周
需求分析与系统设计						
基础平台开发						
前后端联调						
AI 模块集成						
测试与文档整理						

2.7 里程碑

2.7.1 里程碑设置原则

本项目里程碑计划按照“关键阶段完成即形成里程碑”的原则进行设置。每一个里程碑均对应项目实施过程中的一个重要节点，既反映阶段性任务的完成情况，也为后续工作的开展提供前提条件。

2.7.2 具体里程碑设置

里程碑编号	里程碑名称	完成时间	主要内容
M1	需求分析与系统设计完成	第 3 周末	完成需求细化、功能划分、数据库设计和接口设计
M2	基础平台开发完成	第 5 周末	完成商家端、用户端核心业务模块的主要开发
M3	前后端主链路联调完成	第 6 周末	完成注册登录、菜品浏览、下单等核心流程联调
M4	AI 点餐助手集成完成	第 7 周末	完成大模型接口接入及推荐功能实现
M5	项目测试与最终提交完成	第 8 周结课前	完成系统测试、问题修复、文档整理与项目提交

2.7.3 里程碑作用

设置上述里程碑，可以将整个项目实施过程划分为若干可检查、可控制的阶段，有助于及时掌握项目进度，发现偏差并进行调整。从而使项目计划更加清晰，实施过程更具可控性。